UT2. PROGRAMACIÓN MULTIHILO

ACTIVIDAD 6

Sincronización entre hilos (semaphore)

Título: Acceso a sala de videojuegos usando semáforos en Java.

Descripción:

En esta actividad, simularás el acceso a una sala de videojuegos en línea donde un número limitado de jugadores puede unirse al juego al mismo tiempo. El objetivo es garantizar que solo un número máximo de jugadores esté en la sala en cada momento utilizando la clase Semaphore, gestionando el acceso de manera eficiente y segura en un entorno multihilo.

Objetivos:

- Comprender cómo funciona la clase Semaphore para gestionar la sincronización multihilo.
- Controlar el acceso limitado a recursos compartidos en aplicaciones multihilo.
- Implementar y ejecutar un programa Java que simule un entorno realista de concurrencia utilizando semáforos.

Instrucciones:

1. **Preparación del proyecto:**
   - Crea un proyecto llamado **SimulacionSalaVideojuegos** con una clase llamada **SalaVideojuegos**.
2. **Declaraciones iniciales:**
   - Declara en SalaVideojuegos una constante static final int PLAZAS = 4.
   - Declara un atributo static Semaphore salaDisponible = new Semaphore(PLAZAS). Esto simula que solo 4 jugadores pueden estar en la sala a la vez.
3. **Clase Jugador:**
   - Dentro de SalaVideojuegos, crea la clase **static class Jugador**, que implemente Runnable. Debe ser static para poder crear jugadores desde main.
   - Su constructor recibe el número del jugador: public Jugador(int numero).
   - En run(), en este orden:
     1. Muestra Jugador X quiere entrar.
     2. Pide un permiso con salaDisponible.acquire(). Si no quedan permisos, el hilo espera.
     3. Muestra Jugador X ENTRA (en sala: N), donde N es PLAZAS - salaDisponible.availablePermits().
     4. Simula el tiempo de juego con Thread.sleep(new Random().nextInt(2000, 5001)), es decir, entre 2 y 5 segundos.

5. Muestra Jugador X SALE y, después, libera el permiso con salaDisponible.release().
   - acquire() y sleep() lanzan InterruptedException: captúrala con try/catch en run().
4. **Simulación:**
   - En main, muestra tu nombre. Después, con un bucle del 1 al 10, crea 10 hilos, cada uno con un Jugador con su número, e inícialos con start().
5. **Ejecución y análisis:**
   - Ejecuta el programa y comprueba que el número de jugadores en sala nunca pasa de 4.
   - Cambia PLAZAS a 5 y observa cómo cambia la ejecución.
   - **Experimento:** vuelve a 4 plazas y comenta la línea del release(). Ejecuta y observa qué pasa. Para el programa con el botón rojo de IntelliJ y después vuelve a dejar el release().

Ten en cuenta: el número "en sala" es una foto del momento en que se consulta, y dos jugadores que entran casi a la vez pueden mostrar el mismo número. Lo importante es que nunca supera el número de plazas.

Rúbrica:

0	No se cumple ninguno de los requisitos mínimos.
3	El programa no usa el semáforo o no compila.
5	Usa el semáforo, pero no se respeta el límite o algún permiso no se libera.
7	Simulación correcta, pero falta la captura o la explicación.
9	Todo correcto: código, captura y explicación razonada.
10	Todo correcto y, además, incluye una mejora que aporta valor al programa.

Entrega:

i. Incluye aquí el código fuente:

```
import java.util.Random;
import java.util.concurrent.Semaphore;
public class SalaVideojuegos {

   static final int PLAZAS = 4;
   static Semaphore salaDisponible = new Semaphore(PLAZAS);

   static class Jugador implements Runnable {
      private int numero;

      public Jugador(int numero) {
         this.numero = numero;
      }

      @Override
      public void run() {
         try {
            System.out.println("Jugador " + numero + " quiere entrar");
```

```
            salaDisponible.acquire();

            int enSala = PLAZAS - salaDisponible.availablePermits();
            System.out.println("Jugador " + numero + " entra en sala: " + enSala);

            Thread.sleep(new Random().nextInt(2000, 5001));

            System.out.println("Jugador " + numero + " SALE");

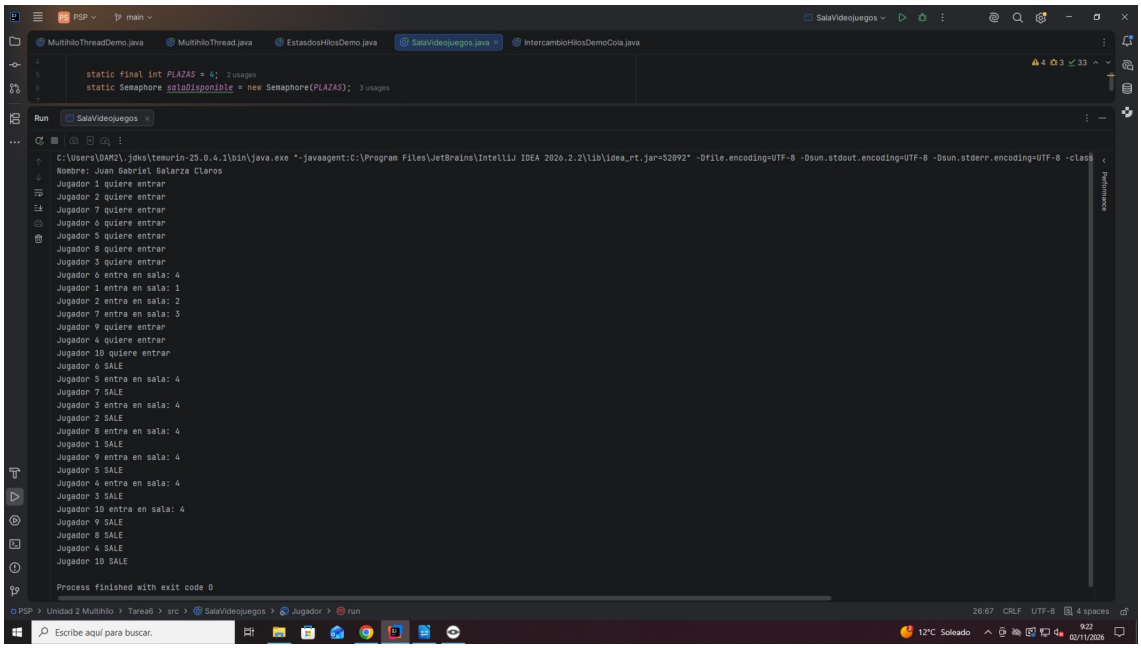
        } catch (InterruptedException e) {
            System.out.println("El jugador " + numero + " fue interrumpido");
            Thread.currentThread().interrupt();
        } finally {
            // aporte para el 10
            // colocar release() en un finally garantiza que el permiso siemper
            // se devuelva al semaforo, incluso si el hilo falla o se interrumpe por error
            salaDisponible.release();
        }
    }
}

public static void main(String[] args) {
    System.out.println("Nombre: Juan Gabriel Galarza Claros");

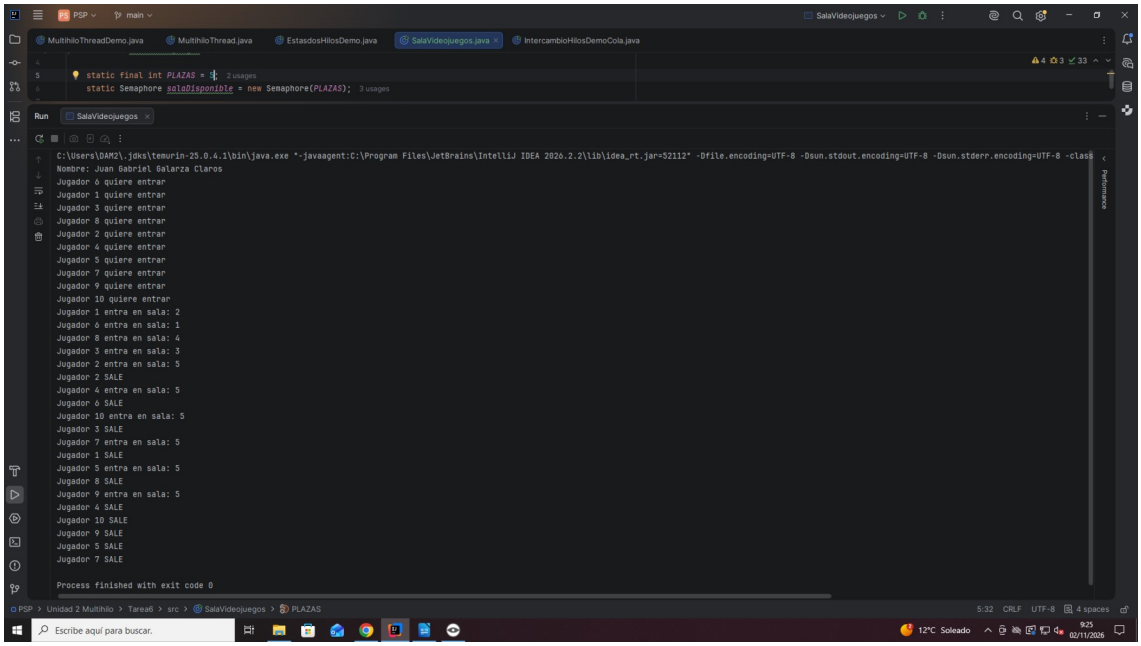
    // Bucle para iniciar y crear  10 hilos
    for (int i = 1; i <= 10; i++) {
        Thread hilo = new Thread(new Jugador(i));
        hilo.start();
    }
}
}
```

ii. Incluye aquí la captura de pantalla de la consola con los mensajes de entrada y salida de los jugadores, donde se vea tu nombre:

CAPTURA CON NÚMERO PLAZAS = 4



CAPTURA CON NÚMERO PLAZAS = 5



iii. Explica brevemente qué ocurrió en el experimento sin release() y cómo controla Semaphore el acceso a la sala:

EXPLICACIÓN:

Al quitar el release(), los primeros 4 jugadores entran a la sala, pero cuando terminan de jugar no liberan su plaza. Esto hace que el contador del semáforo se quede en 0 para siempre. El resultado es que los otros 6 jugadores se quedan esperando infinitamente y el programa se queda pillado, porque nadie avisa de que hay huecos libres